Python Programming **Name:Afrose A**
Reg no:1924260309

ASSIGNMENT 1

CO1: *Apply syntax and semantics of Python programming to develop unstructured coding using the interpreter, values, types, variables, expressions, functions and modules – interactive and script modes, data types (int, float, bool, str, list), variables, expressions, statements, operators, functions, modules and algorithmic problem solving.*

Assignment Title: Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K3 – Apply, K4 – Analyse, K5 – Evaluate (the level of each task is stated under the task heading)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA08-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Python source files (.py or .ipynb), input files and output screenshots.
- Include a README.md with your name, register number, Python version (for example Python 3.12), the libraries used with their versions (requirements.txt), the command used to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/afroseafrose/CSA-0802-1924260309-assignment1

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Ksheera Village Milk Producers' Co-operative

Twice a day, about 300 farmers bring milk to the Ksheera co-operative collection centre. For each farmer, the clerk records the quantity in litres, the fat percentage and the SNF (solids-not-fat) percentage. The price per litre is Rs 30.00 for milk with 3.5% fat and 8.5% SNF; every 0.1% of fat above or below 3.5% adds or subtracts Rs 0.40, and every 0.1% of SNF above or below 8.5% adds or subtracts Rs 0.30. Farmers are paid every ten days.

The present calculator, written quickly by a volunteer, gives totals that differ from the bank statement by a few paise, rejects some valid entries, and has the rates typed in several places, so that a rate revision was applied only partly.

The volume of data may grow:

1 centre (300 farmers) → 1 block (6,000 farmers) → 1 district union (150,000 farmers)

You are required to analyse the problem, choose suitable values, types, expressions, functions and modules, implement the calculator in Python and validate it, so that every payment is correct to the paisa.

Design Challenge:

The co-operative is installing an automatic milk analyser and wants seasonal rate revisions.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

Introduction and Given Scenario

The Ksheera Village Milk Producers' Co-operative records litres, fat percentage and SNF percentage for each farmer. The standard rate is Rs. 30.00 per litre at 3.5% fat and 8.5% SNF. Each 0.1% fat step changes the rate by Rs. 0.40 and each 0.1% SNF step changes it by Rs. 0.30. Payments are calculated every ten days. Decimal arithmetic is recommended for accurate money calculations.

```
fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)
rate = 30.00 + fat_steps * 0.40 + snf_steps * 0.30
amount = litres * rate
```

TASK 1 – Understand the Problem and Identify the Data

Data	Type	Purpose
Farmer ID	str/int	Identifies the farmer; str supports IDs with leading zeros or letters.
Litres	Decimal	Milk quantity; fractional litres are allowed.
Fat and SNF	Decimal	Milk quality values used to calculate the rate.
Shift	str	Morning or evening collection.
Rate and amount	Decimal	Price per litre and payment for an entry.
Ten-day total	Decimal	Sum of a farmer's payments during ten days.

Constants: BASE_RATE=30.00, STANDARD_FAT=3.5, STANDARD_SNF=8.5, FAT_STEP=0.40, SNF_STEP=0.30, STEP_SIZE=0.1, MIN_FAT=2.0 and MAX_FAT=7.0. Keep these in one place so a rate revision is simple.

Litres cannot be negative.

Fat must be between 2.0% and 7.0%, inclusive.

SNF must be numeric and non-negative; the PDF does not specify an upper limit.

Farmer ID and shift should be recorded for identification and auditing.

Assumption: each entry is rounded to two decimal places before adding it to the ten-day total.

Alternative: separate variables such as litres1, fat1, snf1 through 300 entries are difficult to maintain. A list of tuples or records can be processed with a loop. Processing n entries takes O(n) time. A list uses O(n) space; separate variables also need O(n) space but do not scale conveniently.

TASK 2 – Explore Values and Types in the Interpreter

```
>>> type(10)       # <class 'int'>
>>> type(10.0)     # <class 'float'>
>>> type("10")     # <class 'str'>
>>> type(True)     # <class 'bool'>
>>> type([10, 8.5])# <class 'list'>
>>> 7 / 2          # 3.5
>>> 7 // 2         # 3
>>> 7 % 2          # 1
>>> -7 // 2        # -4
>>> 0.1 + 0.2 == 0.3  # False
```

The / operator gives true division, // gives floor division, and % gives the remainder. -7 // 2 is -4 because floor division rounds down. 0.1 and 0.2 cannot be represented exactly in binary floating-point, so their sum is not exactly 0.3.

Completed TODO 1:

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Expected output given in the PDF: float : 33400.30000000061; Decimal: 33400.30; False True.

Decimal(amount) converts the original string exactly. Decimal(float(amount)) first converts to an inexact binary float, so the Decimal can contain extra digits. Integer paise is another exact option. Rounding floats after every addition is not guaranteed to be exact. Summing n amounts is O(n) time; a running total is O(1) extra space.

TASK 3 – Choose Variables and Name the Constants

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
STEP_SIZE = 0.1
MIN_FAT = 2.0
MAX_FAT = 7.0

farmer_id = "F101"
litres = 10.5
shift = "Morning"
a, b = 10, 20
a, b = b, a
print(a, b)
```

Variables normally use lowercase_with_underscores and constants use UPPER_CASE by convention. Python does not enforce constants; programmers avoid changing uppercase

names. Python is dynamically typed: x=10 makes x an int, then x='ten' makes it a str. Multiple assignment swaps values without a temporary variable.

Magic numbers repeated in k places require k edits for a rate revision. Named constants require one edit. A RATE_CARD dictionary such as {'base':30.0,'fat':0.40,'snf':0.30} is also suitable. Runtime and memory differences are negligible; maintainability is the main advantage.

TASK 4 – Write Correct Expressions

```
fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)
rate = 30.00 + fat_steps * 0.40 + snf_steps * 0.30
amount = litres * rate
```

Parentheses are evaluated first, then multiplication/division, then addition/subtraction. Exponentiation has higher precedence than unary minus: -2**2 is -4, while (-2)**2 is 4. Most operators of equal precedence associate left to right; exponentiation associates right to left.

```
>>> (4.2 - 3.5) / 0.1   # 7.000000000000001
>>> 0.1 + 0.2 == 0.3    # False
>>> -7 // 2             # -4
>>> int(-3.9)           # -3
>>> import math
>>> math.floor(-3.9)    # -4
```

The first result is floating-point approximation. round() chooses the nearest integer; int() truncates toward zero; math.floor() rounds down. For fat=4.2 the intended step count is +7; for fat=3.1 it is -4. Using int() can truncate a value slightly below +7 to 6 or a value near -4 toward -3. floor() can also change the intended count if the floating approximation lies just below an integer. Use Decimal-based step calculation for reliable readings. Each operation is O(1) time and O(1) space.

The chained condition 2.0 <= fat <= 7.0 checks both limits. Boolean operators and, or and not combine conditions.

TASK 5 – Read Input and Convert Types

```
text = input("Enter litres: ")
try:
    litres = float(text.strip())
    print(f"Quantity: {litres:8.2f}")
except ValueError:
    print("Invalid number. Enter a value such as 8.5.")
```

input() always returns a string. int() converts integer text and float() converts decimal text. '8,5' and 'eight' raise ValueError with float(). Surrounding spaces in ' 8.5 ' are accepted after strip(). The format {amount:8.2f} prints a value in a field of width 8 with two decimal places.

The try/except approach is clear and handles invalid text safely. Conversion and validation take O(1) time per ordinary entry. A simple pre-check can reject some strings but try/except handles actual numeric conversion more reliably.

TASK 6 – Organise the Logic into Functions

```
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP = Decimal("0.40")
SNF_STEP = Decimal("0.30")

def rate_per_litre(fat, snf):
    """Return the rate per litre."""
    fs = int(((Decimal(str(fat))-Decimal("3.5"))/Decimal("0.1"))
            .quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    ss = int(((Decimal(str(snf))-Decimal("8.5"))/Decimal("0.1"))
            .quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    return (BASE_RATE + fs*FAT_STEP + ss*SNF_STEP).quantize(Decimal("0.01"))

def payout(litres, fat, snf):
    """Return payment for one entry."""
    return (Decimal(str(litres))*rate_per_litre(fat, snf)).quantize(Decimal("0.01"))

def ten_day_total(entries):
    """Return total payment for all entries."""
    total = Decimal("0.00")
    for litres, fat, snf in entries:
        total += payout(litres, fat, snf)
    return total.quantize(Decimal("0.01"))

print(ten_day_total([(10,3.5,8.5),(8.5,4.2,8.7)]))
```

rate_per_litre() accepts fat and SNF and returns the rate. payout() accepts litres, fat and SNF and returns one payment. ten_day_total() accepts a list of entries and returns the total. return gives a value to the caller; print only displays it. Constants can be read inside functions but should not be reassigned. Local variables exist only within their function.

A long script repeats code and is harder to test. Lambda is suitable for short expressions, not multi-step calculations. Function calls add small overhead, but processing n entries remains O(n).

TASK 7 – Build a Module

Save the following as milkrate.py:

```
"""Milk co-operative rate module."""
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")

def rate_per_litre(fat, snf):
    """Return price per litre rounded to two decimals."""
    fs = int(((Decimal(str(fat))-Decimal("3.5"))/Decimal("0.1"))
            .quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    ss = int(((Decimal(str(snf))-Decimal("8.5"))/Decimal("0.1"))
            .quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    rate = BASE_RATE + fs*FAT_STEP_VALUE + ss*SNF_STEP_VALUE
    return rate.quantize(Decimal("0.01"))
```

```
def payout(litres, fat, snf):
    """Return amount payable rounded to two decimals."""
    return (Decimal(str(litres))*rate_per_litre(fat,snf)).quantize(Decimal("0.01"))

if __name__ == "__main__":
    for litres,fat,snf in [(10,3.5,8.5),(8.5,4.2,8.7),(12,3.1,8.3)]:
        rate = rate_per_litre(fat,snf)
        pay = payout(litres,fat,snf)
        print(f"{litres:5} L fat {fat} snf {snf} rate {rate:6.2f} pay {pay:8.2f}")
```

Save this second file as test_module.py in the same folder:

```
import milkrate

litres = 8.5
fat = 4.2
snf = 8.7

rate = milkrate.rate_per_litre(fat, snf)
amount = milkrate.payout(litres, fat, snf)
print("Rate per litre:", rate)
print("Payment:", amount)
```

import milkrate imports the module; use milkrate.payout(). from milkrate import payout imports the function directly. import milkrate as mr gives an alias; use mr.payout(). The if __name__ == '__main__' block runs only when the module is executed directly, not when imported, so its sample table is not printed on import. Python searches the current folder and sys.path. Keep both files in the same folder. Standard modules include decimal and math.

Expected direct output: 10 L / rate 30.00 / pay 300.00; 8.5 L / rate 33.40 / pay 283.90; 12 L / rate 27.80 / pay 333.60. A shared module avoids copying functions into several scripts. Import has a one-time loading cost and Python caches the module in sys.modules.

TASK 8 – Plan the Algorithm Before Coding

Pseudocode

```
START
SET total = 0
FOR each milk entry
    READ farmer ID, litres, fat, SNF and shift
    IF litres < 0 OR fat < 2.0 OR fat > 7.0 OR SNF < 0
        DISPLAY "Invalid entry"
    ELSE
        fat_steps = nearest_integer((fat - 3.5) / 0.1)
        snf_steps = nearest_integer((SNF - 8.5) / 0.1)
        rate = 30 + fat_steps*0.40 + snf_steps*0.30
        amount = round(litres*rate, 2)
        total = total + amount
        DISPLAY rate and amount
    END IF
END FOR
DISPLAY total
STOP
```

Flowchart in text

START -> Read litres, fat, SNF -> Inputs valid?
No -> Display error -> Read next entry
Yes -> Calculate rate -> Calculate payment
-> Add payment to total -> More entries?
Yes -> Read next entry
No -> Display ten-day total -> STOP

Hand trace

Entry	Litres	Fat	SNF	Rate Rs.	Payment Rs.
1	10	3.5	8.5	30.00	300.00
2	8.5	4.2	8.7	33.40	283.90
3	12	3.1	8.3	27.80	333.60
Total					917.50

A list-based solution takes O(n) time and O(n) space. A running total takes O(n) time and O(1) extra space. Store all entries when an audit trail, correction or later report is required.

TASK 9 – Implement and Test the Calculator

```
from decimal import Decimal, ROUND_HALF_UP

def rate_per_litre(fat, snf):
    fs = int(((Decimal(str(fat))-Decimal("3.5"))/Decimal("0.1"))
             .quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    ss = int(((Decimal(str(snf))-Decimal("8.5"))/Decimal("0.1"))
             .quantize(Decimal("1"), rounding=ROUND_HALF_UP))
    return (Decimal("30.00")+fs*Decimal("0.40")+ss*Decimal("0.30")).quantize(Decimal("0.01"))

def payout(litres, fat, snf):
    if Decimal(str(litres)) < 0:
        raise ValueError("Negative quantity")
    if not Decimal("2.0") <= Decimal(str(fat)) <= Decimal("7.0"):
        raise ValueError("Fat out of range")
    if Decimal(str(snf)) < 0:
        raise ValueError("Invalid SNF")
    return (Decimal(str(litres))*rate_per_litre(fat,snf)).quantize(Decimal("0.01"))

def run_tests():
    assert payout(10,3.5,8.5) == Decimal("300.00")
    assert payout(8.5,4.2,8.7) == Decimal("283.90")
    assert payout(12,3.1,8.3) == Decimal("333.60")
    assert rate_per_litre(2.0,8.5) == Decimal("24.00")
    assert rate_per_litre(7.0,8.5) == Decimal("44.00")
    try:
        payout(-1,3.5,8.5)
        assert False
    except ValueError:
        pass
    try:
        payout(10,7.5,8.5)
        assert False
    except ValueError:
        pass
    print("All tests passed.")

run_tests()
```

Test	Input	Expected result
Standard	10 L, fat 3.5, SNF 8.5	Rs. 300.00
Above standard	8.5 L, fat 4.2, SNF 8.7	Rs. 283.90
Below standard	12 L, fat 3.1, SNF 8.3	Rs. 333.60
Minimum fat	fat 2.0	Rate Rs. 24.00
Maximum fat	fat 7.0	Rate Rs. 44.00
Negative litres	-1 L	ValueError
Fat out of range	fat 7.5	ValueError
Text input	eight	Catch ValueError and ask again

The three sample entries total Rs. 917.50. Add two more entries to complete the PDF's requested five-entry manual total. Automated assert tests can be rerun quickly after each code change. With m tests, the test suite takes O(m) time.

TASK 10 – Compare Expectations with Execution

```
from decimal import Decimal
import time

for n in (300, 6000, 150000):
    values = ["33.40"] * n

    start = time.perf_counter()
    float_total = sum(float(x) for x in values)
    float_time = time.perf_counter() - start

    start = time.perf_counter()
    decimal_total = sum((Decimal(x) for x in values), Decimal("0.00"))
    decimal_time = time.perf_counter() - start

    print(n, float_total, decimal_total, float_time, decimal_time)
```

Exact totals for repeated Rs. 33.40 payments are Rs. 10,020.00 for 300 payments, Rs. 200,400.00 for 6,000 and Rs. 5,010,000.00 for 150,000. Float may show tiny differences; Decimal created from strings is exact. Run the code and record the actual times from your own computer because timings depend on hardware and system load; the PDF does not provide benchmark times.

Rounding each payment to paisa before adding matches a system that pays each entry to the nearest paisa. Rounding only the final total can differ if individual values contain fractions of a paisa. The co-operative must select one official policy and apply it consistently.

A dictionary can cache each (fat, snf) rate. Repeated pairs can use average O(1) lookup instead of recalculation. A cache for k distinct pairs uses O(k) extra space.

TASK 11 – Recommend and Defend the Final Design

Recommendation: retain the modular design, using Decimal for money and decimal readings, named constants for the rate card, validation functions, and a list or database of farmer entries.

Round each payable amount to two decimal places using a documented rule such as ROUND_HALF_UP.

Change the rate in one constants section or external rate-card file.

Separate functions for validation, rate calculation, payout and ten-day total make the program easier to test.

Processing n entries takes O(n) time; storing entries takes O(n) space.

For 150,000 farmers, use a database for persistent records, indexing, backups and reports while reusing the Python calculation module.

Limitations include incorrect clerk/analyser readings, calibration errors, rate-policy changes and the unspecified maximum SNF limit.

A JSON or text rate card can be read when the program starts. Reading t values takes O(t) time and O(t) space. It adds a small startup cost but avoids editing and redistributing the program for every rate revision. Validate the file before using it.

```
import json
with open("rate_card.json", "r") as file:
    rate_card = json.load(file)

# rate_card.json:
# {"base": "30.00", "fat_step": "0.40", "snf_step": "0.30"}
```

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Challenge)

A. Impact Analysis

The analyser now sends fat and SNF to two decimal places. From April to June the base rate becomes Rs. 32.00 and each fat step is Rs. 0.45. The SNF step remains Rs. 0.30 because no change is specified. A farmer supplying more than 500 litres in ten days receives a bonus of Rs. 0.50 per litre. The rate constants, rate-selection function and ten-day total function must be updated. This solution rounds two-decimal readings to the nearest 0.1 percentage-point step.

B. Evaluation of Original Design

The original module and functions remain suitable, so modify rather than completely redesign. Decimal can store the analyser readings. A season parameter selects the correct rate card and a separate bonus calculation handles the new condition. At fat 4.2 and SNF 8.7, the summer rate is 32.00 + 7(0.45) + 2(0.30) = Rs. 35.75 per litre; for 10 litres the payment is Rs. 357.50 before bonus.

C. Adaptation – Revised Code

```
from decimal import Decimal, ROUND_HALF_UP

STEP = Decimal("0.1")
def steps(value, standard):
    return int(((Decimal(str(value))-Decimal(standard))/STEP)
               .quantize(Decimal("1"), rounding=ROUND_HALF_UP))
```

```
def seasonal_rate(fat, snf, season="normal"):
    fs = steps(fat, "3.5")
    ss = steps(snf, "8.5")
    if season == "summer":      # April to June
        base, fat_value = Decimal("32.00"), Decimal("0.45")
    else:
        base, fat_value = Decimal("30.00"), Decimal("0.40")
    rate = base + fs*fat_value + ss*Decimal("0.30")
    return rate.quantize(Decimal("0.01"))

def ten_day_payment(entries, season="normal"):
    total_litres = sum(Decimal(str(e[0])) for e in entries)
    total = Decimal("0.00")
    for litres, fat, snf in entries:
        amount = Decimal(str(litres))*seasonal_rate(fat,snf,season)
        total += amount.quantize(Decimal("0.01"))
    bonus = Decimal("0.00")
    if total_litres > Decimal("500"):
        bonus = (total_litres*Decimal("0.50")).quantize(Decimal("0.01"))
    return total, bonus, total+bonus

sample = [(10,4.2,8.7), (520,3.5,8.5)]
print("Normal:", ten_day_payment(sample,"normal"))
print("Summer:", ten_day_payment(sample,"summer"))
```

D. Evidence-Based Defence

Example	Calculation	Result
Normal: 10 L, fat 4.2, SNF 8.7	30 + 7×0.40 + 2×0.30	Rate Rs. 33.40; payment Rs. 334.00
Summer: 10 L, fat 4.2, SNF 8.7	32 + 7×0.45 + 2×0.30	Rate Rs. 35.75; payment Rs. 357.50
Bonus for 520 L	520 × 0.50	Rs. 260.00
Exactly 500 L	Bonus applies only above 500 L	No bonus

The revised design supports both seasons without duplicating the calculation. Bonus is added only when the ten-day supply is strictly greater than 500 litres. This interpretation applies Rs. 0.50 to every litre supplied by a qualifying farmer, based on the PDF wording. The co-operative should confirm this policy before deployment.